

Database System

数据库系统

Instructor: Yiwen Shen (沈轶闻), Ph.D.
Dept. of Software & Computer Engineering,
Ajou University, Korea
chrisshen@ajou.ac.kr

今日提纲

- Review
 - 基本 SQL 命令
 - 中级 SQL 命令
- 高级 SQL

高级 SQL

大纲

- 从编程语言访问 SQL
- 函数和程序
- 触发器
- 递归查询
- 高级聚合功能

从编程语言访问 SQL

- 数据库程序员必须能够使用通用编程语言，至少有两个原因：
 - 并非所有查询都可以用 SQL 表达，因为 **SQL 不提供通用语言的全部表达能力**。
 - **非声明性操作**（例如打印报告、与用户交互或将查询结果发送到图形用户界面）无法在 SQL 中完成。

从编程语言访问 SQL (续)

- 有两种方法可以从通用编程语言访问 SQL
- **动态 SQL :**
一种通用程序可以使用一组函数连接数据库服务器并与之通信
- **嵌入式 SQL:** 提供程序与数据库服务器交互的方式。
 - SQL 语句在编译时被转换为**函数调用**。
 - 在运行时, 这些函数调用使用**提供动态 SQL 功能的 API** 连接到数据库。

JDBC (Java Database Connectivity)

- JDBC 是一种用于与支持 SQL 的数据库系统进行通信的 Java API。
- JDBC 支持多种查询和更新数据以及检索查询结果的功能。
- JDBC 还支持元数据检索，例如查询数据库中存在的关系以及关系属性的名称和类型。
- 与数据库通信的模型：
 - 打开连接
 - 创建一个“**语句**”对象
 - 使用语句对象执行查询以发送查询并获取结果
 - 处理错误的异常机制

JDBC 代码

```
public static void JDBCexample(String dbid, String userid, String passwd) {  
    try (Connection conn =  
        DriverManager.getConnection("jdbc:oracle:thin:@db.yale.edu:2000:univdb",  
            userid, passwd);  
        Statement stmt = conn.createStatement(); )  
    {  
        ... Do Actual Work ...  
    }  
    catch (SQLException sqle) {  
        System.out.println("SQLException : " + sqle);  
    }  
}
```

注意：以上语法适用于 Java 7 和 JDBC 4 及以上版本。

以“try (....)”语法 (“try with resources”) 打开的资源会在 try 块结束时自动关闭。

旧版本 Java/JDBC 的 JDBC 代码

```
public static void JDBCexample(String dbid, String userid, String passwd) {  
    try {  
        Class.forName ("oracle.jdbc.driver.OracleDriver");  
        Connection conn =  
DriverManager.getConnection("jdbc:oracle:thin:@db.yale.edu:2000:univdb", userid,  
passwd);  
        Statement stmt = conn.createStatement();  
        ... Do Actual Work ...  
        stmt.close();  
        conn.close();  
    }  
    catch (SQLException sqle) {  
        System.out.println("SQLException : " + sqle);  
    }  
}
```

注意：从 JDBC 4 开始， `Class.forName` 不再是必需的。上一张幻灯片中的 `try with resources` 语法是 Java 7 及更高版本的首选。

JDBC 代码 (续)

- 更新数据库

```
try {  
    stmt.executeUpdate("insert into instructor values('77987', 'Kim',  
    'Physics', 98000)");  
}  
catch (SQLException sqle){  
    System.out.println("Could not insert tuple. " + sqle);  
}
```

- 执行查询并获取并打印结果

```
ResultSet rset = stmt.executeQuery("select dept_name, avg (salary)  
                                     from instructor  
                                     group by dept_name");  
  
while (rset.next()) {  
    System.out.println(rset.getString("dept_name") + " " + rset.getFloat(2));  
}
```

JDBC 子部分

- 连接数据库
- 将 SQL 语句发送到数据库系统
- 异常和资源管理
- 检索查询结果
- 准备好的语句
- 可调用语句
- 元数据功能
- 其他功能

JDBC 代码详细信息

- 获取结果字段:

如果dept_name是select的结果的第一个参数, 则

`rset.getString("dept_name")` 和 `rs.getString(1)` 等效。

- 处理 Null 值

```
int a = rs.getInt("a");  
if (rs.isNull()) Systems.out.println("Got null value");
```

准备好的声明

```
PreparedStatement pstmt = conn.prepareStatement("insert into instructor  
values(?,?,?,?)");  
pstmt.setString(1, "88877");  
pstmt.setString(2, "Perry");  
pstmt.setString(3, "Finance");  
pstmt.setInt(4, 125000);  
pstmt.executeUpdate();  
pstmt.setString(1, "88878");  
pstmt.executeUpdate();
```

- 警告：在从用户那里获取输入并将其添加到查询时，始终使用准备好的语句
 - 切勿通过连接字符串来创建查询
 - "insert into instructor values(' " + ID + " ', ' " + name + " ', " + "' + dept name + " ', " + " ' balance + ') " "
 - 如果名字是“ D'Souza ”怎么办？

SQL注入

- 假设查询是使用

```
"select * from instructor where name = ' " + name + " ' "
```

- 假设用户不输入姓名，而是输入：

- 'X' or 'Y' = 'Y'

- 那么结果语句就变成：

```
"select * from instructor where name = ' " + "X' or 'Y' = 'Y'" + " ' "
```

- 即： `select * from instructor where name = 'X' or 'Y' = 'Y';`

- 用户甚至可以使用

- `X'; update instructor set salary = salary + 10000;`

在内部使用准备好的语句：

```
"select * from instructor where name = 'X\' or \'Y\' = \'Y\'"
```

- 始终使用准备好的语句，并将用户输入作为参数

元数据功能 Metadata

- **ResultSet**元数据
- 例如执行查询后得到一个ResultSet rs :

```
ResultSetMetaData rsmd = rs.getMetaData();  
for(int i = 1; i <= rsmd.getColumnCount(); i++) {  
    System.out.println(rsmd.getColumnName(i));  
    System.out.println(rsmd.getColumnTypeName(i));  
}
```

- 这有什么用处?

元数据 (续)

- 数据库元数据

```
DatabaseMetaData dbmd = conn.getMetaData();
// Arguments to getColumns: Catalog, Schema-pattern, Table-pattern,
// and Column-Pattern
// Returns: One row for each column; row has a number of attributes
// such as COLUMN_NAME, TYPE_NAME
// The value null indicates all Catalogs/Schemas.
// The value "" indicates current catalog/schema
// The value "%" has the same meaning as SQL like clause
ResultSet rs = dbmd.getColumns(null, "univdb", "department", "%");
while( rs.next()) {
    System.out.println(rs.getString("COLUMN_NAME"),
                       rs.getString("TYPE_NAME"));
}
```

- 这有什么用处?

元数据 (续)

- 数据库元数据

```
DatabaseMetaData dbmd = conn.getMetaData();
// Arguments to getTables: Catalog, Schema-pattern, Table-pattern,
// and Table-Type
// Returns: One row for each table; row has a number of attributes
// such as TABLE_NAME, TABLE_CAT, TABLE_TYPE, ..
// The value null indicates all Catalogs/Schemas.
// The value "" indicates current catalog/schema
// The value "%" has the same meaning as SQL like clause
// The last attribute is an array of types of tables to return.
// TABLE means only regular tables
ResultSet rs = dbmd.getTables("", "", "%", new String[] {"TABLES"});
while( rs.next()) {
    System.out.println(rs.getString("TABLE_NAME"));
}
```

- 这有什么用处?

查找主键

```
DatabaseMetaData dmd = connection.getMetaData();  
// Arguments below are: Catalog, Schema, and Table  
// The value "" for Catalog/Schema indicates current catalog/schema  
// The value null indicates all catalogs/schemas  
ResultSet rs = dmd.getPrimaryKeys("", "", tableName);  
while(rs.next()){  
    // KEY_SEQ indicates the position of the attribute in  
    // the primary key, which is required if a primary key has multiple  
    // attributes  
    System.out.println(rs.getString("KEY_SEQ"),rs.getString("COLUMN_NAME"));  
}
```

JDBC 中的事务控制

- 默认情况下，每个 SQL 语句都被视为一个单独的事务，并自动提交
 - 对于具有多个更新的事务来说，这是一个坏主意
- 可以关闭连接的自动提交
 - `conn.setAutoCommit (false);`
- 然后必须明确提交或回滚事务
 - `conn.commit ();`
 - `conn.rollback();`
- `conn.setAutoCommit (true)`开启自动提交。

其他 JDBC 特性

- 调用函数和过程

```
CallableStatement cStmt1 = conn.prepareCall("{? = call some function(?)})";  
CallableStatement cStmt2 = conn.prepareCall("{call some procedure(?,?)})";
```

- 处理大型对象类型

- `getBlob ()` 和 `getClob ()` 与 `getString ()` 方法类似, 但分别返回 `Blob` 和 `Clob` 类型的对象
- `getBytes ()` 从这些对象中获取数据
- 将打开的流与 Java `Blob` 或 `Clob` 对象关联起来以更新大对象

```
blob.setBlob(int parameterIndex, InputStream inputStream)
```

JDBC 资源

- JDBC 基础教程

- <https://docs.oracle.com/javase/tutorial/jdbc/index.html>

SQLJ

- JDBC 过于动态，编译器无法捕获错误

- **SQLJ** : Java 中的嵌入式 SQL

```
#sql iterator deptInfoIter ( String dept_name, int avgSal);  
deptInfoIter iter = null;  
#sql iter = { select dept_name, avg(salary) from instructor  
              group by dept_name };  
while (iter.next()) {  
    String deptName = iter.dept_name();  
    int avgSal = iter.avgSal();  
    System.out.println(deptName + " " + avgSal);  
}  
iter.close();
```

ODBC

- **开放数据库连接 (ODBC) 标准**
 - 应用程序与数据库服务器通信的标准。
 - 应用程序接口 (API)
 - 打开与数据库的连接,
 - 发送查询和更新,
 - 得到结果。
- **GUI、电子表格等应用程序可以使用 ODBC**

嵌入式 SQL

- SQL 标准定义了 SQL 在各种编程语言（如 C、C++、Java、Fortran 和 PL/1）中的嵌入，
- 嵌入 SQL 查询的语言称为**宿主语言**，宿主语言中允许的 SQL 结构构成**嵌入式SQL**。
- 这些语言的基本形式遵循 System R 将 SQL 嵌入到 PL/1 中。
- **执行 SQL** 语句在宿主语言中用于向预处理器标识嵌入式 SQL 请求
EXEC SQL <embedded SQL statement >;

注意：这因语言而异：

- 在某些语言中，例如 COBOL，分号被替换为 END-EXEC
- 在 Java 中嵌入使用 #SQL { };

嵌入式 SQL (续)

- 在执行任何 SQL 语句之前，程序必须首先连接到数据库。具体操作如下：

EXEC SQL connect to server user *user-name* using *password*;

这里，*server* 识别要建立连接的服务器。

- 宿主语言的变量可以在嵌入式 SQL 语句中使用。它们前面带有冒号 (:)，以区别于 SQL 变量（例如：*credit_amount*）。
- 上述使用的变量必须在 **DECLARE** 部分中声明，如下所示。声明变量的语法遵循通常的宿主语言语法。

EXEC SQL BEGIN DECLARE SECTION

int *credit-amount* ;

EXEC SQL END DECLARE SECTION;

嵌入式 SQL (续)

- 要编写嵌入式 SQL 查询，我们使用

declare c cursor for <SQL query>

语句。变量 **c** 用于标识查询

- 例子：
 - 在宿主语言中，找到学生的 ID 和姓名，这些学生完成了超过 **credit_amount** 变量中存储的学分数
 - 在 SQL 中指定查询如下：

```
EXEC SQL
    declare c cursor for
        select ID, name
        from student
        where tot_cred > :credit_amount
END_EXEC
```

嵌入式 SQL (续)

- 示例的 **open** 语句如下：

EXEC SQL open c ;

此语句使数据库系统执行查询并将结果保存在临时关系中。查询使用宿主语言中的变量 *credit-amount* 值在open命令被执行时。

- **fetch** 语句引起查询结果中一个元组的值被放置在宿主语言变量上。

EXEC SQL

fetch c into :si, :sn

END_EXEC

重复调用 **fetch** 会在查询结果中获取连续的元组

嵌入式 SQL (续)

- SQL 通信区域 (SQLCA) 中名为 SQLSTATE 的变量被设置为“02000”，表示没有更多可用数据
- **close** 语句导致数据库系统删除保存查询结果的临时关系。

EXEC SQL close c ;

注意：以上细节因语言而异。

例如，Java 嵌入定义了 Java 迭代器来遍历结果元组。

通过嵌入式 SQL 进行更新

- 用于数据库修改（更新、插入和删除）的嵌入式 SQL 表达式
- 可以通过声明用于更新的光标来更新通过光标获取的元组

```
EXEC SQL
  declare c cursor for
    select *
    from instructor
    where dept_name = 'Music'
    for update
```

- 然后，我们通过对游标执行提取操作来遍历元组（如前所述），提取每个元组后，我们执行以下代码：

```
update instructor
set salary = salary + 1000
where current of c
```

在 Python 中使用 SQL

- 通过在 Python 中导入不同的模块，我们可以连接数据库并发送 SQL 查询。
- SQLite3: `import sqlite3`
- MySQL: `python -m pip install mysql-connector-python`
`import mysql.connector`
- PostgreSQL: `pip install psycopg2`
If doesn't work, use "pip install psycopg2-binary"
`import psycopg2`

Python 中的 SQLite3

```
import sqlite3
con = sqlite3.connect("univdb-sqlite.db")

cur = con.cursor()

res = cur.execute("SELECT * FROM course;")

ret = res.fetchall()

print(ret)
```

Python 中的 SQLite3

```
import sqlite3
con = sqlite3.connect("univdb-sqlite.db")

cur = con.cursor()

cur.execute("""
    INSERT INTO course
    VALUES ("BDS-101", "Database System", "BDS", 3);
""")

con.commit()

res = cur.execute("SELECT * FROM course;")
ret = res.fetchall()
print(ret)
```

INSERT

语句隐式打开一个事务，需要在将更改保存到数据库之前提交该事务

在 SQL 查询中使用占位符绑定值

```
con = sqlite3.connect(":memory:")
cur = con.execute("CREATE TABLE lang(name, first_appeared)")

# This is the named style used with executemany():
data = (
    {"name": "C", "year": 1972},
    {"name": "Fortran", "year": 1957},
    {"name": "Python", "year": 1991},
    {"name": "Go", "year": 2009},
)
cur.executemany("INSERT INTO lang VALUES(:name, :year)", data)

# This is the qmark style used in a SELECT query:
params = (1972,)
cur.execute("SELECT * FROM lang WHERE first_appeared = ?", params)
print(cur.fetchall())
con.close()
```

对于命名样式，参数必须是字典（或子类）的实例，它必须包含所有命名参数的键；任何额外的项目都将被忽略。

对于 **qmark style**，参数必须是一个序列，其长度必须与占位符的数量匹配，否则会引发 `ProgrammingError`。

函数和过程

Functions and Procedures

函数和过程

- **函数**和**过程**允许将“业务逻辑”存储在数据库中并通过 **SQL** 语句执行。
- 这些可以由 **SQL** 的过程组件或外部编程语言（如 **Java**、**C** 或 **C++**）定义。
- 我们在此处介绍的语法由 **SQL** 标准定义。
 - 大多数数据库都实现了该语法的非标准版本。

声明 SQL 函数

- 定义一个函数，给定一个部门的名称，返回该部门的教师人数。

```
create function dept_count (dept_name varchar(20))  
returns integer  
begin  
declare d_count integer;  
    select count(*) into d_count  
    from instructor  
    where instructor.dept_name = dept_name  
return d_count;  
end
```

- 函数 *dept_count* 可用于查找所有拥有 12 名以上教师的部门的名称和预算。

```
select dept_name, budget  
from department  
where dept_count (dept_name ) > 12;
```

表函数

- SQL 标准支持可以返回表作为结果的函数；这样的函数称为表函数
- 示例：返回给定部门的所有讲师

```
create function instructor_of (dept_name char(20))
  returns table (
    ID          varchar(5),
    name        varchar(20),
    dept_name    varchar(20),
    salary       numeric(8,2))
  return table (
    select ID, name, dept_name, salary
    from instructor
    where instructor.dept_name = instructor_of.dept_name);
```

- 用法

```
select * from table (instructor_of('Music'));
```

SQL 过程

- *department* 计数函数也可以写成过程：

```
create procedure dept_count_proc (in dept_name varchar(20),  
out d_count integer)  
begin  
    select count(*) into d_count  
    from instructor  
    where instructor.dept_name = dept_count_proc.dept_name  
end;
```

- 关键字 **in** 和 **out** 分别是
 - in: 是会被分配值的参数
 - out: 是在过程中被分配值以返回结果的参数
- 可以使用调用语句从 SQL 过程或嵌入式 SQL 调用过程。

```
declare d_count integer;  
call dept_count_proc( 'Physics', d_count);
```

SQL 过程 (续)

- 也可以从动态 SQL 调用过程和函数
- SQL 允许有多个过程，只要同名过程的参数个数不同即可。
- 名称以及参数的数量用于识别过程。

过程和函数的语言结构

- SQL 支持的结构使其几乎具备通用编程语言的所有功能。
 - 警告: 大多数数据库系统都实现了以下标准语法的变体。
- 复合语句: **begin ... end** ,
 - begin和end之间可能包含多个 SQL 语句。
 - 局部变量可以在复合语句中声明
- While 和 repeat 语句:
 - **while** boolean expression **do**
sequence of statements ;
end while
 - **repeat**
sequence of statements ;
until boolean expression
end repeat

语言结构 (续)

- **For**循环
 - 允许迭代查询的所有结果
- 示例：查找所有部门的预算

```
declare n integer default 0;
for r as
    select budget from department
    where dept_name = 'Music'
do
    set n = n + r.budget
end for
```

语言结构——if-then-else

- 条件语句 (if-then-else)

```
if boolean expression
    then statement or compound statement
elseif boolean expression
    then statement or compound statement
else statement or compound statement
end if
```

示例程序

- 确保教室容量不超过规定后，注册学生
 - 成功时返回 0, 超出容量时返回 -1
- 发出异常情况信号并声明异常处理程序

```
declare out_of_classroom_seats condition
declare exit handler for out_of_classroom_seats
begin
...
end
```

- **begin**和**end**之间的语句可以通过执行“**signal out_of_classroom_seats**”
- 处理程序表示，如果出现这种情况，要采取的行动是退出 **begin end** 语句。

外部语言常规

- SQL 允许我们使用 Java、C#、C 或 C++ 等编程语言定义函数。
 - 比SQL中定义的函数更高效, 并且可以由这些函数执行SQL中无法执行的计算。
- 声明外部语言过程和函数

```
create procedure dept_count_proc(in dept_name varchar(20),  
                                out count integer)
```

```
language C
```

```
external name '/usr/avi/bin/dept_count_proc'
```

```
create function dept_count(dept_name varchar(20))
```

```
returns integer
```

```
language C
```

```
external name '/usr/avi/bin/dept_count'
```

外部语言常规 (续)

- 外部语言函数/过程的好处：
 - 许多操作更加高效, 表达能力更强。
- 缺点
 - 实现功能的代码可能需要加载到数据库系统中并在数据库系统的地址空间中执行。
 - 数据库结构意外损坏的风险
 - 安全风险, 允许用户访问未经授权的数据
 - 还有其他替代方案, 它们可以提供良好的安全性, 但性能可能会变差。
 - 当效率比安全性更重要时, 使用直接在数据库系统执行过程。

使用外部语言常规的安全性

- 为了解决安全问题，我们可以采取以下措施：
 - 使用沙盒技术
 - 也就是说，使用像Java这样的安全语言，不能用它来访问/破坏数据库代码的其他部分。
 - 在单独的进程中运行外部语言函数/过程，无需访问数据库进程的内存。
 - 通过进程间通信传递参数和结果
- 两者都有性能开销
- 许多数据库系统支持上述两种方法以及直接在数据库系统地址空间中执行。

触发器

Triggers

触发器

- **触发器 (trigger)** 是数据库修改时系统自动执行的语句。
- 要设计触发机制，我们必须：
 - 指定触发器执行的**条件**。
 - 指定触发器执行时要采取的**操作**。
- 触发器在 SQL:1999 中引入 SQL 标准，但大多数数据库甚至更早就使用非标准语法支持触发器。
 - 这里说明的语法**可能不完全适用于**一些数据库系统;请查看各自数据库系统手册

在 SQL 中触发事件和操作

- 触发事件可以是插入、删除或更新
- 更新触发器可以限制为特定属性
 - 例如, 在 *选课* 关系的 *成绩* 属性更新后
- 可以引用更新前后的属性值
 - **referencing old row as** : 用于删除和更新
 - **referencing new row as** : 用于插入和更新
- 触发器可以在事件发生之前激活, 作为额外的约束。例如, 将空白成绩转换为空值。

```
create trigger setnull_trigger before update of takes
referencing new row as nrow
for each row
    when (nrow.grade = ' ')
    begin atomic
        set nrow.grade = null;
    end;
```

触发维持 学分值

```
create trigger credits_earned after update of takes on (grade)
referencing new row as nrow
referencing old row as orow
for each row
when nrow.grade <> 'F' and nrow.grade is not null
    and (orow.grade = 'F' or orow.grade is null)
begin atomic
    update student
    set tot_cred= tot_cred +
        (select credits
         from course
         where course.course_id= nrow.course_id)
    where student.id = nrow.id;
end;
```

语句级触发器 (statement trigger)

- 无需对每个受影响的行执行单独的操作，而是可以对受事务影响的所有行执行单个操作
 - 使用 **for each statement** 而不用 for each row
 - 使用 **referencing old table** 或 **referencing new table** 引用包含受影响行的临时表（称为**过渡表 transition tables**）
 - 处理更新大量行的 SQL 语句时效率更高

何时不使用触发器

- 触发器以前用于执行以下任务：
 - 维护**汇总数据**（例如，每个部门的总工资）
 - 通过**记录特殊关系**（称为**变更**或**增量**关系）的更改来**复制数据库**，并有一个单独的进程将**更改应用到副本**
- 也有更好的方法可以实现这些目标：
 - 当今的数据库提供内置的**物化视图**功能来维护摘要数据
 - 数据库提供对**复制**的内置支持
- 在许多情况下，可以使用**封装设施 (Encapsulation facilities)** 代替触发器
 - 定义更新字段的方法
 - 将操作作为更新方法的一部分执行，而不是通过触发器

何时不使用触发器 (续)

- 意外执行触发器的风险，例如，当
 - 从备份副本加载数据
 - 在远程站点复制更新
 - 可以在执行此类操作之前禁用触发器执行。
- 触发因素的其他风险：
 - 导致触发关键事务失败的错误
 - 级联执行

递归查询

Recursive Queries

SQL中的递归查询

- SQL:1999 允许递归视图定义 (**with recursive**)
- 使用递归查找哪些课程是特定课程的先修课程，无论是直接还是间接

```
with recursive rec_prereq(course_id, prereq_id) as (  
    select course_id, prereq_id  
    from prereq  
    union  
    select rec_prereq.course_id, prereq.prereq_id  
    from rec_prereq, prereq  
    where rec_prereq.prereq_id = prereq.course_id  
)  
select *  
from rec_prereq;
```

- 这个示例视图 *rec_prereq* 被称为 *prereq* 的 **传递闭包 (transitive closure)** 关系

递归的力量

- 递归视图使得编写查询, 例如**传递闭包查询**, 成为可能, 而如果没有递归或迭代就无法编写这些查询。
 - 直观感受: 如果没有递归, 非递归非迭代程序只能执行**固定数量**的 *prereq* 与自身的连接
 - 这只能提供**固定数量**的管理人员级别
 - 给定一个固定的非递归查询, 我们可以构建一个具有更多层级的先决条件的数据库, 查询在这些层级上将不起作用
 - 替代方案: 编写一个函数/过程 (function/procedure) 来根据需要进行多次迭代
 - 之后的 *findAllPrereqs* 程序

递归的有利方面

- 使用迭代计算传递闭包，将连续元组添加到 *rec_prereq*
 - 下一张幻灯片展示了 *prereq* 关系
 - 迭代过程的每一步都会构建一个 *rec_prereq* 的扩展版本从其递归的定义。
 - 最终结果称为递归视图定义的 **不动点(fixed point)**。
- 递归视图 **必须是单增的**。
 - 也就是说，如果我们将元组添加到 *prereq* 中，则视图 *rec_prereq* 包含之前包含的所有元组，以及可能更多的元组
- 任何递归视图都必须定义为 **两个子查询的并集**：一个非递归的基本查询和一个使用递归视图的递归查询。

定点 (fixed point) 计算示例

<i>course_id</i>	<i>prereq_id</i>
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-190
CS-319	CS-101
CS-319	CS-315
CS-347	CS-319

<i>Iteration Number</i>	<i>Tuples in c1</i>
0	
1	(CS-319)
2	(CS-319), (CS-315), (CS-101)
3	(CS-319), (CS-315), (CS-101), (CS-190)
4	(CS-319), (CS-315), (CS-101), (CS-190)
5	done

```

create function findAllPrereqs(cid varchar(8))
-- Finds all courses that are prerequisite (directly or indirectly) for cid
returns table (course_id varchar(8))
-- The relation prereq(course_id, prereq_id) specifies which course is
-- directly a prerequisite for another course.
begin
create temporary table c_prereq (course_id varchar(8));
-- table c_prereq stores the set of courses to be returned
create temporary table new_c_prereq (course_id varchar(8));
-- table new_c_prereq contains courses found in the previous iteration
create temporary table temp (course_id varchar(8));
-- table temp is used to store intermediate results
insert into new_c_prereq
select prereq_id
from prereq
where course_id = cid;
repeat
insert into c_prereq
select course_id
from new_c_prereq;

insert into temp
(select prereq.prereq_id
 from new_c_prereq, prereq
 where new_c_prereq.course_id = prereq.course_id
 )
except (
select course_id
from c_prereq
);
delete from new_c_prereq;
insert into new_c_prereq
select *
from temp;
delete from temp;

until not exists (select * from new_c_prereq)
end repeat;
return table c_prereq;
end

```

另一种方法: 使用函数(function)

高级聚合功能

Advanced Aggregation Features

排名 Ranking

- 排名是按照规范顺序进行的。
- 假设我们有一个关系 *student_grades*(ID, GPA)

给出每个学生的平均成绩

- 找出每个学生的排名。

```
select ID, rank() over (order by GPA desc) as s_rank  
from student_grades;
```
- 需要额外的 **order by** 子句来按排序顺序排列它们

```
select ID, rank() over (order by GPA desc) as s_rank  
from student_grades  
order by s_rank;
```

- 排名可能会留下间距：例如，如果 2 名学生的最高 GPA 相同，则两人的排名均为 1，下一个排名为 3
 - 密集等级 没有留下空隙，所以下一个密集等级是 2

排名

- 可以使用基本的 SQL 聚合进行排名，但结果查询效率很低

```
select ID, (1 + (select count(*)  
                from student_grades B  
                where B.GPA > A.GPA)) as s_rank  
from student_grades A  
order by s_rank;
```

排名 (续)

- 可以在数据分区内进行排名。
- “查找每个部门内学生的排名。”

```
select ID, dept_name,  
       rank () over (partition by dept_name order by GPA desc) as dept_rank  
from dept_grades  
order by dept_name, dept_rank;
```

- 单个选择子句中 can 出现多个rank子句。
- 应用group by子句/聚合后 进行排名
- 可用于查找前 n 个结果
 - 比用 limit 更普遍, 许多数据库都支持

排名 (续)

- 其他排名函数：
 - **percent_rank** (如果已进行分区, 则在分区内)
 - **cume_dist** (累积分布)
 - 具有先前值的元组的比例
 - **row_number** (存在重复时不确定)
- **SQL:1999** 允许用户指定第一个空值或最后一个空值

```
select ID, rank() over (order by GPA desc nulls last) as s_rank  
from student_grades;
```

排名 (续)

- 对于给定的常数 n ，排序函数 $ntile(n)$ 按指定的顺序获取每个分区中的元组，并将它们分成 n 个具有相同元组数量的桶。
- 例如，

```
select ID, ntile(4) over (order by GPA desc) as quartile  
from student_grades;
```


窗口化 Windowing

- 用于消除随机变化。
- 例如，移动平均值：“给定每个日期的销售值，计算每个日期当天、前一天和后一天的销售额平均值”
- 窗户规格 在 SQL 中：
 - 给定关系销售(日期, 价值)

```
select date, sum(value) over (order by date between rows 1 preceding  
and 1 following)  
from sales;
```

窗口化

- 其他窗口规格示例：
 - 无界前置行和当前行之间 (between rows unbounded preceding and current)
 - 行无界前 (rows unbounded preceding)
 - 范围介于前10行和当前行之间
 - 值在当前行值 -10 到当前值之间的所有行
 - 范围间隔10天前 (range interval 10 day preceding)
 - 不包括当前行

窗口化 (续)

- 可以在**分区内**进行窗口化
- 例如，给定一个关系 *transaction* (*account_number* , *date_time* , *value*) ，其中value表示存款为正，表示取款为负

```
select account_number, date_time,  
       sum (value) over (partition by account_number  
                        order by date_time  
                        rows unbounded preceding)  
                        as balance  
from transaction  
order by account_number, date_time;
```

在线分析处理

OLAP (Online Analytical Processing)

数据分析和 OLAP

- 在线分析处理 (OLAP)

- 数据的交互式分析, 允许以在线方式(延迟可忽略不计)以不同方式汇总和查看数据

- 可以建模为维度属性和度量属性的数据称为**多维数据**。

- **测量属性**

- 衡量一些价值
 - 可以汇总聚合
 - 例如: *sales* 关系的属性 *number*

- **维度属性**

- 定义查看 度量 属性(或其聚合)的维度
 - 例如, *sales* 关系中的 属性 *item_name*, *color*, 和 *clothes_size*

销售 关系示例

<i>item_name</i>	<i>color</i>	<i>clothes_size</i>	<i>quantity</i>
skirt	dark	small	2
skirt	dark	medium	5
skirt	dark	large	1
skirt	pastel	small	11
skirt	pastel	medium	9
skirt	pastel	large	15
skirt	white	small	2
skirt	white	medium	5
skirt	white	large	3
dress	dark	small	2
dress	dark	medium	6
dress	dark	large	12
dress	pastel	small	4
dress	pastel	medium	3
dress	pastel	large	3
dress	white	small	2
dress	white	medium	3
dress	white	large	0
shirt	dark	small	2
shirt	dark	medium	6

按 *item_name* 和 *color* 交叉制表 *sales* 情况

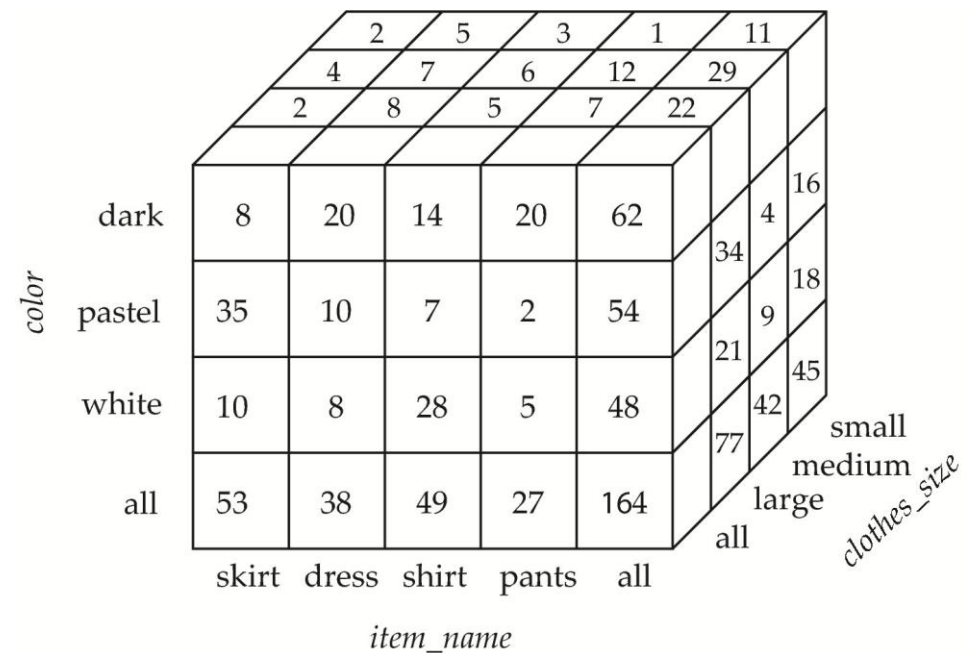
clothes_size **all**

		<i>color</i>			
		dark	pastel	white	total
<i>item_name</i>	skirt	8	35	10	53
	dress	20	10	5	35
	shirt	14	7	28	49
	pants	20	2	5	27
	total	62	54	48	164

- 上面的表格是**交叉表 (cross-tab)** 的示例，也称为**数据透视表 (pivot-table)**。
 - 维度属性之一的值构成**行**标题
 - 另一个维度属性的值构成**列**标题
 - 其他维度属性列在**顶部**
 - 指定该单元格的维度属性的值（的集合）。

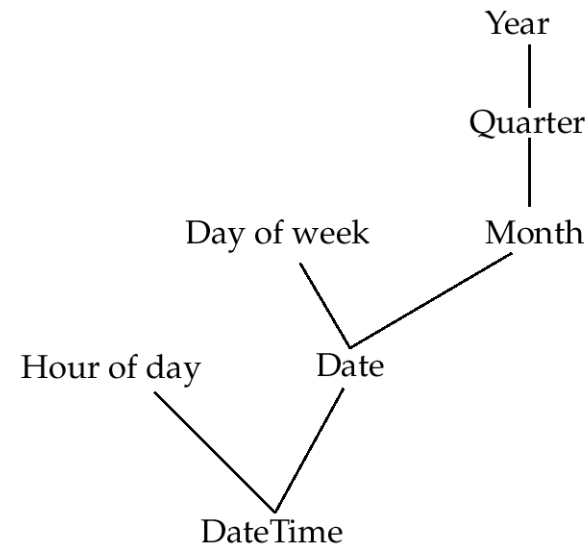
数据立方体 data cube

- 数据立方体 (data cube) 是交叉表的多维概括
- 可以有n 维；我们在下面显示 3维
- 交叉表可以用作数据立方体的视图

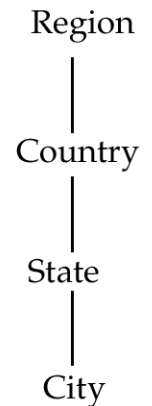


维度上的层次结构

- **层次结构**: 允许以不同的详细程度查看维度
 - 例如, 维度**DateTime**可用于按一天中的小时、日期、一周中的天、月份、季度或年份进行聚合



a) Time Hierarchy



b) Location Hierarchy

具有层次结构的交叉制表

- 交叉表可以轻松扩展以处理层次结构
 - 可以向下钻取 (drill down) 或向上滚动 (rollup) 层次结构

clothes_size: **all**

		<i>color</i>				
<i>category</i>	<i>item_name</i>	dark	pastel	white	total	
womenswear	skirt	8	8	10	53	88
	dress	20	20	5	35	
	subtotal	28	28	15		
menswear	pants	14	14	28	49	76
	shirt	20	20	5	27	
	subtotal	34	34	33		
total		62	62	48		164

交叉表的关系表示

- 交叉表可以表示为**关系**
 - 我们使用值 **all** 来表示**聚合**。
 - 尽管与常规空值相混淆，但 **SQL** 标准实际上使用**空值**代替所有值。

item_name	color	clothes_size	quantity
skirt	dark	all	8
skirt	pastel	all	35
skirt	white	all	10
skirt	all	all	53
dress	dark	all	20
dress	pastel	all	10
dress	white	all	5
dress	all	all	35
shirt	dark	all	14
shirt	pastel	all	7
shirt	White	all	28
shirt	all	all	49
pant	dark	all	20
pant	pastel	all	2
pant	white	all	5
pant	all	all	27
all	dark	all	62
all	pastel	all	54
all	white	all	48
all	all	all	164

扩展聚合以支持 OLAP

- 多维数据集 (cube) 操作 计算指定属性的每个子集上的 **group by** 的并集
- 关系示例: `sales(item_name, color, clothes_size, quantity)`
- 例如, 考虑查询

```
select item_name, color, size, sum(number)
from sales
group by cube(item_name, color, size);
```

销售关系的八个不同分组的并集:

```
{ (item_name, color, size), (item_name, color),
  (item_name, size),      (color, size),
  (item_name),            (color),
  (size),                 () }
```

其中 () 表示一个空的分组列表。

- 对于每个分组, 结果包含分组中不存在的属性的空值。

联机分析处理操作

- 我们之前看到的交叉表的关系表示，但用`null`代替`all`，可以通过以下方式计算

```
select item_name, color, sum(number)
from sales
group by cube(item_name, color);
```

- 函数`grouping()`可以应用于属性
 - 如果值为代表全部的空值则返回 1，其他情况则返回 0。

```
select item_name, color, size, sum(number),
       grouping(item_name) as item_name_flag,
       grouping(color) as color_flag,
       grouping(size) as size_flag
from sales
group by cube(item_name, color, size);
```

在线分析处理操作

- 在select子句中使用函数**decode()**将这些空值替换为all等值
 - 例如, 替换`item_name` 在第一个查询中
`decode( grouping(item_name), 1, 'all', item_name)`

扩展聚合 (续)

- **rollup**构造在指定属性列表的每个前缀上生成并集

- 例如,

```
select item_name, color, size, sum(number)
from sales
group by rollup(item_name, color, size);
```

生成四个分组的并集:

```
{ (item_name, color, size), (item_name, color), (item_name), ( ) }
```

- **Rollup** 可用于在
层次结构的多个级别生成聚合。
- 例如, 假设表 `itemcategory ( item_name , category )` 给出了每个项目的类别。那么

```
select category, item_name, sum(number)
from sales, itemcategory
where sales.item_name = itemcategory.item_name
group by rollup(category, item_name);
```

*item_name*给出分层摘要 并按类别。

扩展聚合 (续)

- 可以在单个 group by 子句中使用多个汇总和多维数据集
 - 每个都生成一组分组列表, 集合的交叉积给出一组整体分组列表
- 例如:

```
select item_name, color, size, sum(number)
from sales
group by rollup(item_name), rollup(color, size);
```

生成分组

$$\{item_name, ()\} \times \{(color, size), (color), ()\}$$
$$= \{ (item_name, color, size), (item_name, color), (item_name), (color, size), (color), () \}$$

联机分析处理操作

- **旋转 Pivoting** : 改变交叉表中使用的维度称为
- **切片 Slicing** : 仅为固定值创建交叉表
 - 有时称为“切块”，特别是当多个维度的值固定时。
- **汇总 Rollup** : 从细粒度数据转移到粗粒度数据
- **向下钻取 Drill down** : 相反的操作 - 从粗粒度数据移动到细粒度数据

OLAP 实现

- 最早的OLAP系统使用内存中的多维数组来存储数据立方体，被称为**多维OLAP (MOLAP)** 系统。
- 仅使用关系数据库功能的 OLAP 实现称为**关系 OLAP (ROLAP)**系统
- 混合系统将一些摘要存储在内存中，将基础数据和其他摘要存储在关系数据库中，称为**混合 OLAP (HOLAP)** 系统。

OLAP 的实现 (续)

- 早期的 OLAP 系统预先计算所有可能的聚合, 以便提供在线响应
 - 这样做的空间和时间要求可能非常高
 - 2^n 个 group by 的组合
 - 只需预先计算一些聚合, 并根据需要从预先计算的聚合之一计算其他聚合即可
 - 可以从 (*item_name* , *color* , *size*) 上的聚合计算出 (*item_name* , *color*) 上的聚合
 - 中位数
 - 比从头计算更小开销
- 可用于计算多个聚合的几种优化
 - 可以从 (*item_name* , *color* , *size*) 上的聚合计算出 (*item_name* , *color*) 上的聚合
 - 基础数据的单一排序来计算 (*item_name* , *color* , *size*) 、 (*item_name* , *color*) 和 (*item_name*) 上的聚合

Database System

数据库系统

Instructor: Yiwen Shen (沈轶闻), Ph.D.

Dept. of Software & Computer Engineering,

Ajou University, Korea

chrisshen@ajou.ac.kr